

Katholieke
Universiteit
Leuven

**Faculty of
Engineering Technology
Ghent**

ADVANCED COMPUTER ARCHITECTURE

Lab 4: Memory Management

Tom Thys

Academic year 2025–2026

Contents

1 Introduction **2**

2 Coalesced Memory Access **2**
2.1 Exercise . . . 2
2.2 Results and Analysis . . . 2
2.2.1 Speedup Factor . . . 2
2.2.2 Execution Time Comparison . . . 2
2.2.3 Performance Gap . . . 2

3 Global, Shared, and Constant Memory for Matrix Multiplication **2**
3.1 Exercise . . . 3
3.2 Kernel Execution Times . . . 3
3.3 Memory Transfer Times . . . 4
3.4 Analysis . . . 4

A CUDA Kernel Implementations **6**
A.1 Coalesced Memory Access Kernels . . . 6
A.2 Matrix Multiplication Kernels . . . 6

1 Introduction

In high-performance parallel computing, particularly in GPU programming with CUDA, performance bottlenecks rarely lie in raw computational power. Instead, they are overwhelmingly driven by memory access patterns and the efficient utilization of the memory hierarchy. Inefficient global memory operations can easily saturate memory bandwidth, causing processing cores (SMs) to stall as they wait for data. This phenomenon, known as memory-bound execution, negates the benefits of parallel execution.

This lab explores two critical techniques for mitigating memory bottlenecks and achieving optimal GPU performance:

1. Ensuring *coalesced memory access* to maximize the efficiency of global memory transactions.
2. Strategically leveraging the various levels of the GPU memory hierarchy: *Global, Shared, and Constant memory*, to exploit data locality and reuse.

2 Coalesced Memory Access

The CUDA execution model groups threads into warps of 32. When a warp accesses global memory, the GPU attempts to satisfy the requests of all 32 threads using the minimum number of memory transactions, a process called memory coalescing. Coalescing is achieved when consecutive threads access consecutive memory locations. This allows a single, wide memory transaction (typically 128 bytes) to fulfill multiple thread requests simultaneously, significantly improving memory bandwidth utilization.

2.1 Exercise

We compare two memory layouts for an image processing kernel (inverting the red channel): the *un-coalesced* format (interleaved RGB) and the *coalesced* format (planar RRR...GGG...BBB...). The coalesced format ensures that neighboring threads within a warp read contiguous bytes from the red channel data, leading to optimal memory transactions. In contrast, un-coalesced access occurs when threads read data from non-contiguous memory locations, leading to multiple memory transactions and slower execution.

2.2 Results and Analysis

The kernels were benchmarked across various thread-per-block configurations and input sizes.

2.2.1 Speedup Factor

The following graph illustrates the speedup achieved by the coalesced kernel compared to the un-coalesced kernel for 1x, 2x, and 4x image sizes.

The graph shows that the speedup is most significant for thread block sizes between 128 and 512. The maximum speedup of roughly 1.76x is achieved with 512 threads for the smallest image size.

2.2.2 Execution Time Comparison

Here we compare the absolute execution times for all configurations.

The coalesced versions (solid lines) consistently show lower execution times compared to their uncoalesced counterparts (dashed lines) for all image sizes.

2.2.3 Performance Gap

This graph highlights the absolute time difference (Uncoalesced - Coalesced) in milliseconds.

The absolute benefit of coalescing generally increases with larger image sizes, as more data is being processed, making the efficiency gains more pronounced.

3 Global, Shared, and Constant Memory for Matrix Multiplication

The GPU memory hierarchy consists of multiple types, each optimized for different access patterns, capacity, and latency. Efficient GPU programs minimize high-latency global memory access by strategically leveraging the faster, on-chip memories.

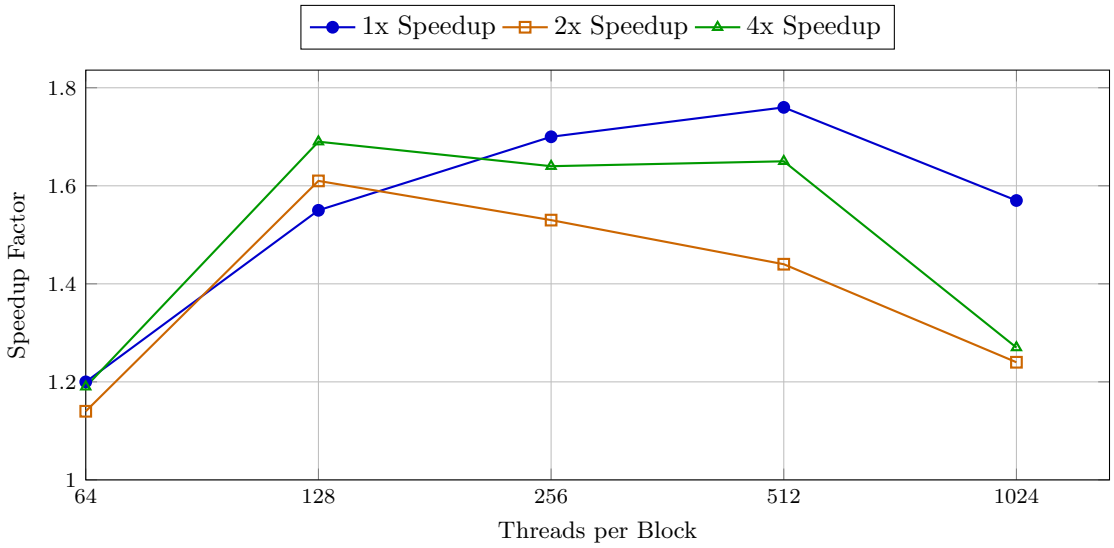


Figure 1: Coalesced Memory Speedup for varying image sizes.

- **Global Memory:**
  - \+ Largest capacity, accessible by all threads across all blocks.
  - – High latency and limited bandwidth; requires coalescing for optimal speed.
- **Shared Memory:**
  - \+ Extremely fast, low-latency memory shared among threads within the same block.
  - \+ Ideal for data reuse and inter-thread communication (requires explicit synchronization).
  - – Limited capacity per SM (typically 48–64KB), with an additional per-block limit. In our experiments, the shared memory usage per block is only a few kilobytes and therefore does not approach this limit.
- **Constant Memory:**
  - \+ Optimized for broadcast reads when all threads in a warp access the same address.
  - – Read-only and small capacity (typically 64KB). Performance degrades significantly when threads within a warp access different addresses, as the broadcast mechanism can no longer be used efficiently.

3.1 Exercise

We implemented matrix multiplication ($C = AB$) using three kernels, each utilizing a different primary memory type: Global-only, Shared (tiled multiplication), and Constant (storing matrix B). We analyze the performance of these kernels as a function of matrix size ($N \times N$) across two block sizes, $BS = 4$ and $BS = 16$.

3.2 Kernel Execution Times

The following tables present the kernel execution times for both block sizes. Note that the Constant Memory approach is capacity-limited.

Table 1: Kernel Execution Times (ms) for Matrix Multiplication ($N \times N$) with $BLOCK_SIZE = 16$

Matrix Size N	**8**	**16**	**32**	**64**	**96**	**192**	**384**	**768**
Global (BS=16)	0.0039	0.0038	0.0046	0.0064	0.0122	0.0534	0.3762	2.8381
Shared (BS=16)	0.0040	0.0043	0.0051	0.0075	0.0152	0.0694	0.4715	3.4653
Constant (BS=16)	0.0040	0.0043	0.0053	0.0092	0.0244	(N/A)	(N/A)	(N/A)

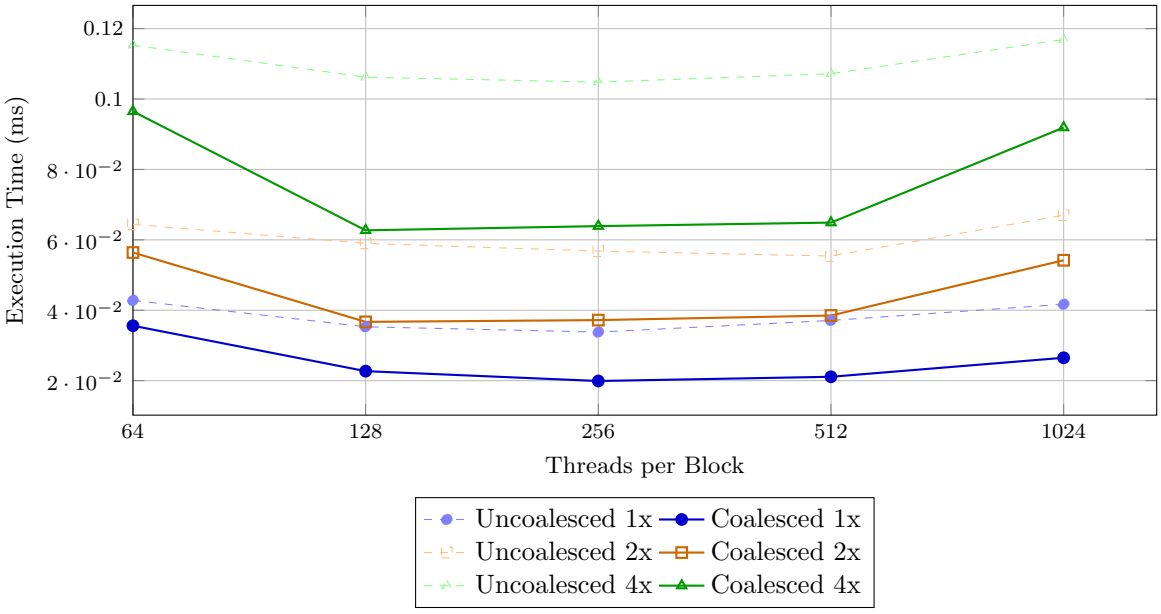


Figure 2: Execution Time vs Thread Configuration across image sizes.

Table 2: Kernel Execution Times (ms) for Matrix Multiplication ($N \times N$) with $BLOCK_SIZE = 4$

Matrix Size N	**8**	**16**	**32**	**64**	**96**	**192**	**384**	**768**
Global (BS=4)	0.0044	0.0046	0.0052	0.0065	0.0094	0.0296	0.1904	1.3779
Shared (BS=4)	0.0046	0.0049	0.0060	0.0086	0.0279	0.1439	0.8480	3.4653
Constant (BS=4)	0.0046	0.0074	0.0120	0.0243	0.0401	(N/A)	(N/A)	(N/A)

3.3 Memory Transfer Times

The following table presents the overhead associated with data movement.

Table 3: Average Memory Transfer Times (ms) from Host (CPU) to Device (GPU)

Matrix Size N	**8**	**16**	**32**	**64**	**96**	**192**	**384**	**768**
Host $\rightarrow$ Device (A)	0.0055	0.0055	0.0060	0.0095	0.0134	0.0261	0.1449	0.4733
Host $\rightarrow$ Device (B)	0.0054	0.0054	0.0060	0.0095	0.0142	0.0234	0.1302	0.6503
Host $\rightarrow$ Constant (B)	0.0026	0.0017	0.0017	0.0017	0.0020	(N/A)	(N/A)	(N/A)

3.4 Analysis

The kernel execution times confirm the theoretical memory hierarchy, but with critical dependence on the implementation details:

- **Shared Memory Performance:** The shared-memory kernel does not consistently outperform the global-memory kernel in our experiments. Although shared memory reduces redundant global memory accesses through tiling, its effectiveness strongly depends on the block size and arithmetic intensity. For $BS = 16$, the shared kernel becomes more competitive due to increased data reuse within each tile, but it remains slightly slower than the global kernel across all tested matrix sizes. For $BS = 4$, the overhead of loading tiles and performing frequent synchronizations dominates the benefits of reuse, leading to significantly worse performance for larger matrices. It is important to note that shared memory capacity is not a limiting factor in these experiments. The shared kernel uses two tiles of size $BLOCK_SIZE \times BLOCK_SIZE$ integers, resulting in only 2KB per block for $BS = 16$ and 128B for $BS = 4$. These values are well below the per-block and per-SM shared memory limits. Consequently, the

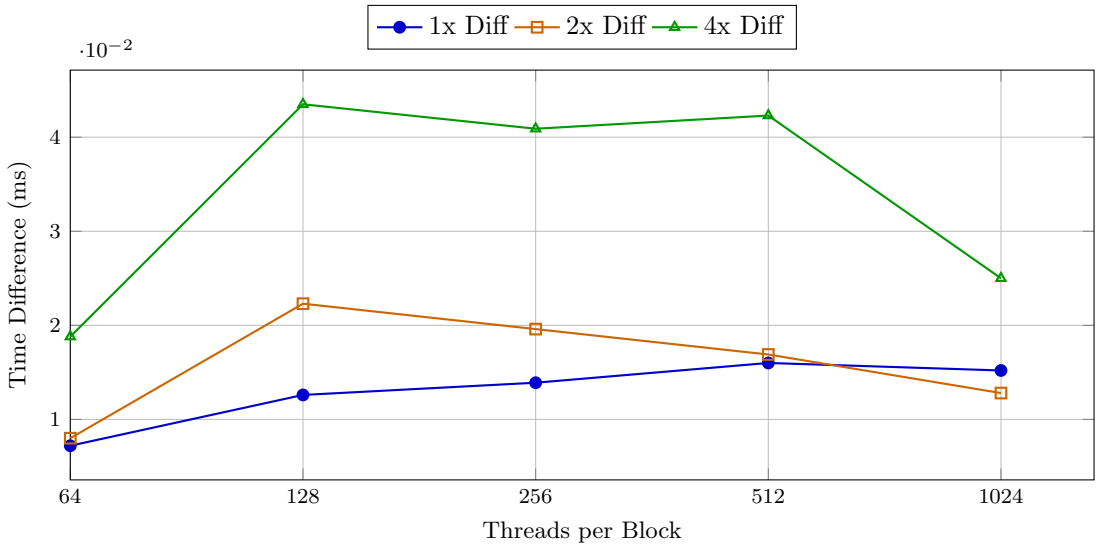


Figure 3: Performance Gap (Uncoalesced - Coalesced) in milliseconds.

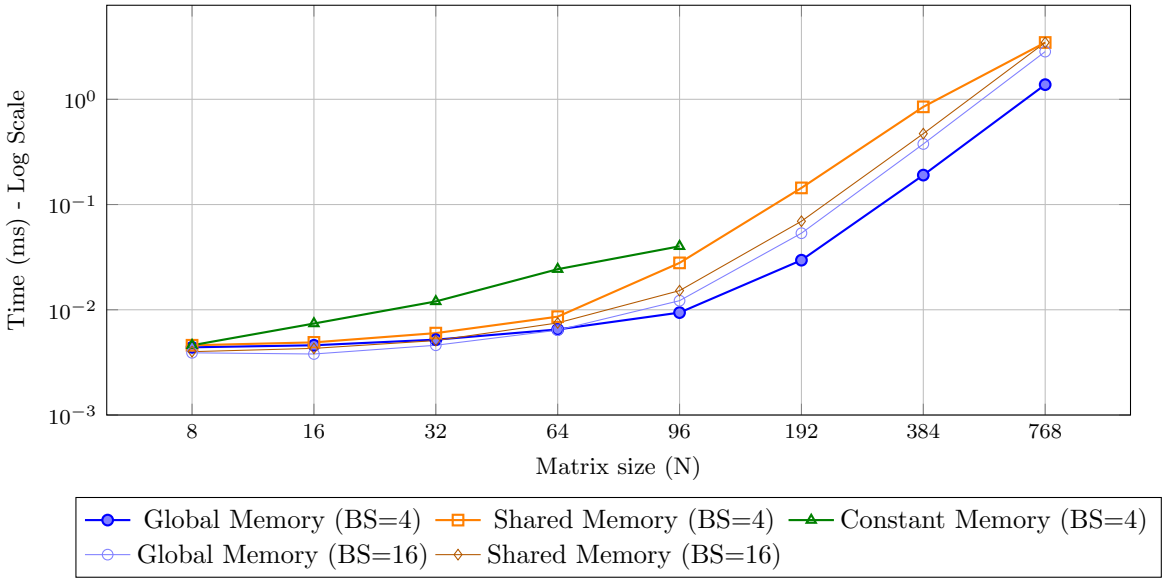


Figure 4: Comparison of kernel execution times (ms) using different memory types across two Block Sizes (Block Size=4 solid, Block Size=16).

observed performance differences are primarily caused by synchronization overhead, block size selection, and limited arithmetic intensity rather than shared memory exhaustion.

- Constant Memory Limitations: Constant memory is extremely fast for small matrices due to its broadcast-optimized cache. This is clearly seen in the very low 'Host $\rightarrow$ Constant' transfer times in Table 3. However, its small capacity (64KB limit) renders it unusable for matrices $N \geq 192$.
- Global Memory Baseline: The simple global-only kernel performs better than both optimized kernels for certain regimes (e.g., $BS = 4$ at large N, and small N generally). This demonstrates that for operations with low computational intensity or small input size, the optimization overhead can outweigh the benefits of using faster memory.

Overall, the results underscore that efficient GPU programming requires minimizing global memory access and leveraging shared or constant memory strategically based on the data access pattern and size constraints.

A CUDA Kernel Implementations

The following listings contain the core CUDA kernels used for memory analysis.

A.1 Coalesced Memory Access Kernels

Listing 1: Coalesced Memory Access Kernel

```
// Operates on a PLANAR array (RRR...GGG...BBB)
__global__ void invert_red_coalesced(uint8_t* planar_input, int num_pixels) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < num_pixels) {
    // Coalesced read/write: consecutive threads access consecutive red pixels
    planar_input[idx] = 255 - planar_input[idx];
  }
}
```

Listing 2: Uncoalesced Memory Access Kernel

```
// Operates on an INTERLEAVED array (RGBRGBRGB...)
__global__ void invert_red_uncoalesced(uint8_t* interleaved_input, int num_pixels) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < num_pixels) {
    int pixel_index = idx * 3;
    // Uncoalesced read/write: consecutive threads access memory locations 3 bytes apart
    interleaved_input[pixel_index] = 255 - interleaved_input[pixel_index];
  }
}
```

A.2 Matrix Multiplication Kernels

Listing 3: Helper Functions for Matrix Access

```
struct Matrix {
  int width, height, stride;
  int* elements;
};

__device int GetElement(const Matrix A, int row, int col) {
  return A.elements[row * A.stride + col];
}

__device void SetElement(Matrix A, int row, int col, int value) {
  A.elements[row * A.stride + col] = value;
}

// Function to get a submatrix/tile (not used by global/constant kernels)
__device Matrix GetSubMatrix(Matrix A, int row, int col) {
  Matrix sub;
  // Note: BLOCK_SIZE is assumed to be defined globally/preprocessor
  sub.width = BLOCK_SIZE;
  sub.height = BLOCK_SIZE;
  sub.stride = A.stride;
  sub.elements = &A.elements[A.stride * BLOCK_SIZE * row + BLOCK_SIZE * col];
  return sub;
}
```

Listing 4: Matrix Multiplication using only Global Memory

```
__global__ void MatMulGlobal(Matrix A, Matrix B, Matrix C) {
  // Use 2D indexing for C(row, col)
  int row = blockIdx.y * blockDim.y + threadIdx.y;
  int col = blockIdx.x * blockDim.x + threadIdx.x;

  if (row >= C.height || col >= C.width) return;

  int sum = 0;
  for (int k = 0; k < A.width; ++k)
```

```
    sum += GetElement(A, row, k) * GetElement(B, k, col);

  SetElement(C, row, col, sum);
}
```

Listing 5: Matrix Multiplication using Shared Memory (Tiling)

```
// Assumes BLOCK_SIZE is defined as a power of 2 (e.g., 4 or 16)
__global__ void MatMulShared(Matrix A, Matrix B, Matrix C) {
  int bx = blockIdx.x, by = blockIdx.y;
  int tx = threadIdx.x, ty = threadIdx.y;

  // Csub is not strictly necessary but keeps the indexing cleaner
  // Matrix Csub = GetSubMatrix(C, by, bx);

  int sum = 0;

  for (int m = 0; m < (A.width / BLOCK_SIZE); ++m) {
    // Load tiles Asub and Bsub into shared memory
    __shared__ int As[BLOCK_SIZE][BLOCK_SIZE];
    __shared__ int Bs[BLOCK_SIZE][BLOCK_SIZE];

    // Determine the global starting index for the current tiles
    int a_start_row = by * BLOCK_SIZE + ty;
    int a_start_col = m * BLOCK_SIZE + tx;
    int b_start_row = m * BLOCK_SIZE + ty;
    int b_start_col = bx * BLOCK_SIZE + tx;

    // Load data from Global Memory to Shared Memory (Coalesced access here is key)
    As[ty][tx] = A.elements[a_start_row * A.stride + a_start_col];
    Bs[ty][tx] = B.elements[b_start_row * B.stride + b_start_col];

    __syncthreads(); // Wait for all threads in the block to load their tile data

    // Perform the matrix multiplication of the shared tiles
    for (int e = 0; e < BLOCK_SIZE; ++e)
      sum += As[ty][e] * Bs[e][tx];

    __syncthreads(); // Wait for all threads to finish using the shared tile
  }

  // Write the final result back to Global Memory
  int globalRow = by * BLOCK_SIZE + ty;
  int globalCol = bx * BLOCK_SIZE + tx;

  if (globalRow < C.height && globalCol < C.width)
    C.elements[globalRow * C.stride + globalCol] = sum;
}
```

Listing 6: Matrix Multiplication using Constant Memory

```
// Assumes matrix B has been copied to the constant memory array `constB`
// The size of matrix B is limited by the constant memory size (64KB)
__constant__ int constB[/* size based on max N */];

__global__ void MatMulConstant(Matrix A, Matrix B, Matrix C) {
  int row = blockIdx.y * blockDim.y + threadIdx.y;
  int col = blockIdx.x * blockDim.x + threadIdx.x;

  if (row >= C.height || col >= C.width) return;

  int sum = 0;
  for (int k = 0; k < A.width; ++k)
    // A is from Global, B is from Constant (optimized broadcast read)
    sum += GetElement(A, row, k) * constB[k * B.width + col];

  SetElement(C, row, col, sum);
}
```